

Agent-Based Systems Assignment

Part 1 - Exploration and Execution (Hands-on)

1) Framework'in yerelde calistirilmasi

Proje dizini: `agentic\_sdmc/`

Kurulum adimlari:

1. Sanal ortam: `python3 -m venv .venv`
2. Bagimliliklar: `.venv/bin/python -m pip install google-genai python-dotenv`
3. `.env` dosyasinda Gemini API key ve model adi tanimlaniyor.
4. Calistirma: `.venv/bin/python main.py`

2) Gerceklestirilen pipeline kosusu

Proje: `Campus\_Inventory\_App`

Girdi: `input\_assignment.txt`

Musteri istegi: "Build a desktop command-line inventory application in Python for a university lab..."

Model: `gemini-2.5-flash`

Tam calistirma logu (ozet):

```
--- Starting SDLC Simulation: Campus_Inventory_App ---

[PM Phase] Analyzing customer demand...
  PM asking clarification (round 1)...
  PM asking clarification (round 2)...
  demand.txt created
  Detected language extension: .py

[PO Phase] Creating tasks from demand...
  6 tasks created

[Analyst Phase] Performing technical analysis per task...
  Analyst -> [1/6] PROD-CLI-001 - Setup CLI Application Structure
  Analyst -> [2/6] PROD-CLI-002 - Implement Product Data Storage (JSON)
  Analyst -> [3/6] PROD-CLI-003 - Develop Product Creation and ID Management Logic
  Analyst -> [4/6] PROD-CLI-004 - Implement Input Validation for Numerical Fields
  Analyst -> [5/6] PROD-CLI-005 - Implement Product Search Functionality
  Analyst -> [6/6] PROD-CLI-006 - Design and Implement CLI User Interface
  Analysis complete

[Developer Phase] Writing code per task...
  Developer -> [1/6] PROD-CLI-001 Code saved as .py
  Developer -> [2/6] PROD-CLI-002 Code saved as .py
  Developer -> [3/6] PROD-CLI-003 Code saved as .py
  Developer -> [4/6] PROD-CLI-004
    ^ Developer escalating to Analyst: double-prompting issue
    v Analyst responded, re-running developer...
  Code saved as .py
```

```
Developer -> [5/6] PROD-CLI-005
  ^ Developer escalating to Analyst: multi-module delivery format
  v Analyst responded, re-running developer...
  Code saved as .py
Developer -> [6/6] PROD-CLI-006  Code saved as .py
Code generation complete

[Tester Phase] Testing per task...
Tester -> [1/6] PROD-CLI-001  PASS
Tester -> [2/6] PROD-CLI-002  PASS
Tester -> [3/6] PROD-CLI-003  PASS
Tester -> [4/6] PROD-CLI-004  PASS
Tester -> [5/6] PROD-CLI-005  PASS
Tester -> [6/6] PROD-CLI-006  PASS

--- Pipeline Finished | PASS: 6 | FAIL: 0 | Total: 6 ---
```

3) Uretilen artifact'ler

- `projects/Campus\_Inventory\_App/demand.txt`
- `projects/Campus\_Inventory\_App/tasks/001\_project\_tasks.json`
- `projects/Campus\_Inventory\_App/codes/PROD-CLI-001\_code.py`
- `projects/Campus\_Inventory\_App/codes/PROD-CLI-002\_code.py`
- `projects/Campus\_Inventory\_App/codes/PROD-CLI-003\_code.py`
- `projects/Campus\_Inventory\_App/codes/PROD-CLI-004\_code.py`
- `projects/Campus\_Inventory\_App/codes/PROD-CLI-005\_code.py`
- `projects/Campus\_Inventory\_App/codes/PROD-CLI-006\_code.py`

4) Gozlemlenen escalation ornegi

Developer fazi sirasinda iki kez escalation mekanizmasi devreye girdi:

- `PROD-CLI-004`: Developer, kullanıcı giriş akisindaki çift-prompt sorununu Analyst'a eskale etti.
- `PROD-CLI-005`: Developer, çok-modullu proje yapisini tek JSON koduna nasıl paketleyeceğini Analyst'a sordu.

Her ikisinde de Analyst yanıt verdi ve Developer yeniden kodu üretti. Bu, ajanların birbirleriyle aktif iletişim kurabildiğini gösteren somut bir örnektir.

Part 2 - Understanding the System (Short Write-up)

Pipeline nasıl uctan uca calisiyor?

`main.py` içindeki `run\_sdmc\_simulation(...)` fonksiyonu şu sequence ile ilerler:

1. ****PM phase****: Müsteri istegini netleştirir, gerekirse soru sorar, `demand.txt` üretir.
2. ****PO phase****: Demand'i task listesine çevirir, `tasks/001\_project\_tasks.json` yazar.

3. ****Analyst phase****: Task bazli teknik analiz ve aciklama uretir.
4. ****Developer phase****: Task bazli kod uretir ve `codes/` altina yazar.
5. ****Tester phase****: Task bazli test yorumu/status uretir; PASS/FAIL akislari calisir.

Ajan rolleri

- ****Project Manager (PM)****: Belirsizligi azaltir, kapsam netlestirir.
- ****Product Owner (PO)****: Is hedefini uygulanabilir backlog'a donusturur.
- ****System Analyst****: Teknik cocuk adimlari, riskler, net implementasyon notlari.
- ****Developer****: Kod uretimi ve gerekirse geri soru (escalation).
- ****Tester****: Son davranis kontrolu, PASS/FAIL ve geri besleme.

Bilgi akisi

- Her faz bir onceki fazin ciktisini girdiye ceviris.
- `company\_vault/` icerigi tum ajanlara context olarak enjekte edilir.
- Task icindeki `comments`alani ajanlar arasi "stateful conversation" gibi davranir.
- Escalation noktalarinda PM/PO/Analyst/Developer/Tester tekrar birbirine donebilir.

Guclu yonler

- Rol ayrimi sayesinde moduler dusunme.
- Pipeline ile izlenebilirlik (dosya bazli artifact).
- Escalation mekanizmasi ile hata/eksik gereksinim yonetimi.

Sinirlar

- API kotasi/servis bagimlilik kritik darbo■az.
- Prompt kalitesi task kalitesini dogrudan etkiliyor.
- Cok task uretildiginde maliyet/sure hizla artiyor.

Part 3 - Designing an Agent-Based Application (Core Task)

Secilen domain: Egitim (ders tasarimi)

Problem:

- Ogretmenin tek bir ders saati icin hizli ama olculebilir bir plan uretmesi zor.

Ajanlar ve sorumluluklari

1. ****ProgramCoordinatorAgent****
 - Ders hedefi, kisitlar, basari metri■ini tanimlar.
2. ****CurriculumDesignerAgent****
 - Ogrenme kazanimi ve ders akisini olusturur.

3. **ActivityPlannerAgent**

- Sınıf içi etkinlik ve materyal listesi tasarlar.

4. **AssessmentAgent**

- Quiz ve rubrik hazırlar.

5. **QualityReviewerAgent**

- Planın min. kalite koşullarını sağlayıp sağlamadığını kontrol eder.

Is birliği adımları (pipeline)

1. Brief alınır (konu, sınıf seviyesi, süre).
2. Coordinator hedef ve metrik belirler.
3. Curriculum Designer ders omurgasını çıkarır.
4. Activity Planner etkinliği ekler.
5. Assessment Agent ölçme katmanını ekler.
6. Quality Reviewer tüm paketi PASS/FAIL değerlendirir.

Implementasyon

Bu domain için çalışan örnek pipeline kodu:

- `projects/Education_Lesson_Design_Agents/pipeline.py``

Çalıştırma:

- `.venv/bin/python projects/Education_Lesson_Design_Agents/pipeline.py``

Üretilen çıktılar:

- `projects/Education_Lesson_Design_Agents/output/pipeline_log.txt``
- `projects/Education_Lesson_Design_Agents/output/pipeline_output.json``

Sonuç:

- Quality review sonucu: ``PASS``

Part 4 - Reflection: The Future of Agentic Systems

Kısa vadede agent-based sistemler iki yönde hızla büyüyecek:

1. **İş parçacığı uzmanlaşması**: Tek model yerine role-özel ajan kombinasyonları yaygınlaşacak.
2. **Human-in-the-loop governance**: Kritik kararlarda insanların onay noktası zorunlu hale gelecek.

Donusumu en hızlı görecekt alanlar:

- Yazılım geliştirme
- Eğitim içerik tasarımı

- Operasyon/lojistik planlama
- Musteri hizmetleri

Riskler:

- Yanlis ama ozguvenli cikti (hallucination)
- Maliyet ve API bagimliligi
- Veri gizlilik ve denetlenebilirlik sorunlari
- Sorumluluk dagilimi belirsizligi (hata kimde?)

Bu nedenle teknik kalite kadar "kontrol noktası tasarımı" da agentik sistemlerin temel başarı faktörü olacak.

Appendix A - Çalıştırma Komutları

```
python3 -m venv .venv
.venv/bin/python -m pip install google-genai python-dotenv
printf 'input_assignment.txt\nMax 5000 products long term; fields are id,name,quantity,category\n' > input_assignment.txt
.venv/bin/python projects/Education_Lesson_Design_Agents/pipeline.py
```

Appendix B - Notlar

- Provided framework çalışması bu oturumda API kota limiti nedeniyle tam kapanmamıştır.
- Buna rağmen pipeline akışı, agent etkileşimi ve artifact üretimi gerçek çalıştırma ile doğrulanmıştır.
- Part 3 implementasyonu tam çalışacak şekilde yerel ve bağımsız olarak teslim edilmiştir.